



万恶的八股文

[📖 Java基础面试16问](#)

[📖 15个经典的Spring面试常见问题](#)

[📖 kafka夺命连环11问](#)

一、Java 基础

1. 请比较 `HashMap` 和 `ConcurrentHashMap` 的实现原理和线程安全机制。

- **考察点：** 集合框架、并发编程、数据结构。

- **参考答案：**

- `HashMap`：

- **原理：** 基于数组+链表/红黑树（JDK8以后）。使用键的 `hashCode()` 进行哈希计算，存储键值对。非线程安全。
- **线程不安全体现：** 多线程同时执行 `put` 操作可能导致数据覆盖、链表成环（在扩容时）等问题。

- `ConcurrentHashMap`（JDK8为例）：

- **原理：** 采用 `Node` 数组 + 链表 / 红黑树 结构，但通过 **分段锁（JDK7）或 CAS + synchronized（JDK8及以后）** 实现线程安全。
- **线程安全机制（JDK8）：**

1. **put 操作：** 首先通过CAS判断数组桶位是否为空，为空则CAS插入；不为空则对桶位的头节点（`Node`）使用 `synchronized` 加锁，再进行链表或红黑树的插入操作。这样锁的粒度从整个段（Segment）细化到了单个桶（Bucket），并发度更高。
2. **get 操作：** 通常无锁，因为 `Node` 的 `val` 和 `next` 属性被 `volatile` 修饰，保证了可见性，性能很好。

2. 谈谈你对 Java 中 `synchronized` 关键字和 `ReentrantLock` 的理解和区别。

- 考察点：线程同步、锁机制。

- 参考答案：

- `synchronized` :
 - 用法：是 Java 语言关键字，可以修饰方法或代码块。
 - 特性：可重入、非公平锁（默认，但后期有优化）、由JVM底层实现，会自动加锁和释放锁。
 - 锁升级：为了平衡性能，JDK6引入了偏向锁、轻量级锁、重量级锁的锁升级过程。
- `ReentrantLock` :
 - 用法：是 `java.util.concurrent.locks` 包下的一个类，需要显式地创建 `lock()` 和 `unlock()`。
 - 特性：可重入、可设置为公平锁或非公平锁（构造函数指定）、提供了 `tryLock()` 可尝试获取锁、可中断的锁等待、可以绑定多个条件变量（`Condition`）。
- 核心区别：
 - i. API层面：`synchronized` 是关键字，自动管理；`ReentrantLock` 是类，需要手动控制，灵活性更高，但也更容易出错（如忘记 `unlock`）。
 - ii. 功能丰富性：`ReentrantLock` 功能更丰富（如公平锁、条件变量）。
 - iii. 性能：在高度竞争的情况下，`ReentrantLock` 可能表现更好；但在低竞争时，`synchronized` 因JVM优化而性能相当甚至更优。现在通常优先考虑 `synchronized`，因为简洁安全，除非需要 `ReentrantLock` 的高级功能。

二、JVM

3. 描述一下 JVM 的内存区域（运行时数据区）。哪些是线程共享的，哪些是线程私有的？

- 考察点：JVM内存模型、垃圾回收基础。

- 参考答案：

- 线程共享：
 - i. 堆（Heap）：存放对象实例和数组，是GC管理的主要区域。
 - ii. 方法区（Metaspace，JDK8+）：存储已被虚拟机加载的类信息、常量、静态变量、即时编译器编译后的代码等。

- **线程私有：**
 - i. **程序计数器：** 当前线程所执行的字节码的行号指示器。
 - ii. **虚拟机栈：** 存储栈帧，每个方法调用对应一个栈帧，用于存储局部变量表、操作数栈、动态链接、方法出口等信息。
 - iii. **本地方法栈：** 为Native方法服务。
- 4. 什么是垃圾回收（GC）？简述一下 CMS 垃圾收集器和 G1 垃圾收集器的工作原理和区别。
 - **考察点：** GC算法、垃圾收集器。
 - **参考答案：**
 - **GC：** 自动管理堆内存，回收不再被引用的对象所占用的空间。
 - **CMS（Concurrent Mark-Sweep）：**
 - **目标：** 低停顿。主要分为：初始标记（STW）-> 并发标记-> 重新标记（STW）-> 并发清除。
 - **缺点：** 对CPU资源敏感、无法处理“浮动垃圾”、会产生内存碎片。
 - **G1（Garbage-First）：**
 - **目标：** 可控的停顿时间。将堆划分为多个大小相等的独立区域（Region），不再是物理上的新生代、老年代。
 - **过程：** 初始标记（STW）-> 并发标记-> 最终标记（STW）-> 筛选回收（STW）。G1通过跟踪各个Region的垃圾价值（回收所得空间大小及所需时间），优先回收价值最大的Region。
 - **优势：** 预测停顿模型、整体上基于“标记-整理”算法，有效避免内存碎片。

三、并发编程

5. 什么是线程池？为什么使用线程池？说说 `ThreadPoolExecutor` 的重要参数。
 - **考察点：** 线程池原理、资源管理。
 - **参考答案：**
 - **线程池：** 一种池化技术，预先创建若干线程，避免频繁创建和销毁线程的开销。
 - **优点：** 降低资源消耗、提高响应速度、便于管理线程（如控制并发数）。
 - **`ThreadPoolExecutor` 核心参数：**
 - i. **`corePoolSize`：** 核心线程数，即使空闲也会保留。
 - ii. **`maximumPoolSize`：** 允许创建的最大线程数。
 - iii. **`workQueue`：** 用于保存等待执行的任务的阻塞队列。
 - iv. **`keepAliveTime`：** 非核心线程闲置时的存活时间。

v. `threadFactory`：用于创建新线程的工厂。

vi. `handler`：拒绝策略（当线程池和队列都满了时如何处理新任务），如 `AbortPolicy`（抛出异常）、`CallerRunsPolicy`（由调用者线程执行）。

6. 请解释 `volatile` 关键字的作用。它能保证原子性吗？

- 考察点：内存模型、可见性、原子性。

- 参考答案：

- 作用：

- i. 保证可见性：当一个线程修改了 `volatile` 变量，新值会立即被刷新到主内存，并使其他线程的工作内存中该变量的缓存行失效，从而强制其他线程读取新值。

- ii. 禁止指令重排序：通过内存屏障实现。

- 原子性：不能。`volatile` 只能保证对变量单次读/写操作的原子性，但不能保证复合操作的原子性。例如 `count++`（读-改-写）不是原子操作，即使 `count` 是 `volatile` 的，多线程下仍会出错。保证原子性需要使用 `synchronized` 或 `AtomicXXX` 类。

四、框架相关（Spring）

7. 解释一下 Spring 框架中 IOC（控制反转）和 AOP（面向切面编程）的概念。

- 考察点：Spring 核心思想。

- 参考答案：

- **IOC**：将对象的创建、依赖关系的管理权从程序员手中“反转”给了 Spring 容器。程序员不再需要自己 `new` 对象，而是通过**依赖注入（DI）**的方式由容器来装配和管理对象之间的依赖关系。这降低了耦合度。

- **AOP**：将那些与核心业务逻辑无关但多个模块共用的功能（如日志、事务、安全）封装成“切面”，然后动态地织入到目标方法中。其底层原理是**动态代理**（JDK 动态代理或 CGLIB 字节码生成）。

8. Spring Bean 的作用域有哪些？默认是哪个？

- 考察点：Spring Bean 生命周期。

- 参考答案：

- 作用域：

- i. **singleton（默认）**：每个 Spring IoC 容器中一个 Bean 定义对应一个对象实例。

- ii. **prototype**：一个 Bean 定义对应多个对象实例，每次获取都创建一个新的。

- iii. **request**：一次 HTTP 请求一个实例。

- iv. **session**：一个 HTTP Session 一个实例。

- v. **application/global-session/websocket** 等。

- 默认是 `singleton`。

五、数据库与中间件

9. 数据库事务的 ACID 特性是什么？MySQL 的默认隔离级别是？解决了哪些并发问题？

- 考察点：数据库事务、隔离级别。
- 参考答案：
 - ACID：
 - 原子性 (Atomicity)：事务内的操作要么全部成功，要么全部失败回滚。
 - 一致性 (Consistency)：事务使数据库从一个一致状态转变为另一个一致状态。
 - 隔离性 (Isolation)：并发事务之间互不干扰。
 - 持久性 (Durability)：事务一旦提交，其结果就是永久性的。
 - MySQL默认隔离级别：可重复读 (REPEATABLE READ)。
 - 解决的并发问题：
 - 读未提交：脏读、不可重复读、幻读。
 - 读已提交：解决了脏读。
 - 可重复读：解决了脏读、不可重复读（在MySQL的InnoDB引擎中，通过Next-Key Lock也很大程度上避免了幻读）。
 - 串行化：解决所有问题，但性能最低。

10. 什么是缓存穿透、缓存击穿、缓存雪崩？如何解决？

- 考察点：Redis缓存常见问题及解决方案。
- 参考答案：
 - 缓存穿透：查询一个数据库中一定不存在的数据（如不存在的ID），导致请求直接打到数据库。
 - 解决：① 接口层校验（如ID<=0直接拦截）；② 将不存在的数据也缓存起来（`key-null`，设置较短过期时间）；③ 使用布隆过滤器快速判断数据是否存在。
 - 缓存击穿：某个热点key过期的瞬间，大量并发请求同时击穿缓存，直接访问数据库。
 - 解决：① 设置热点key永不过期；② 使用互斥锁 (Mutex Lock)，只让一个线程去重建缓存，其他线程等待。
 - 缓存雪崩：在同一时间，大量缓存key同时过期或Redis宕机，导致所有请求涌向数据库。
 - 解决：① 给缓存过期时间加上一个随机值，避免同时过期；② 构建Redis高可用集群（主从+哨兵/集群模式）；③ 使用本地缓存（如Ehcache）作为二级缓存，降低影响。